

# Week 5 & 6: Cleaning Data in Python

Welcome to the week 5 & 6 lab session where we will learn how to diagnose and treat missing data in Python.

Most data science project often revolves around pre-processing data, and making sure data is ready for analysis. In this session, we will be working on how to transform our raw data into accurate insights. In this notebook, you will learn how to:

- Import data into `pandas`, and use simple functions to diagnose problems in our data.
- Visualize missing and out of range data using `missingno` and `seaborn`.
- Apply a range of data cleaning tasks that will ensure the delivery of accurate insights.

## The Dataset Description

The dataset to be used in this session is a CSV file named `airbnb.csv`, which contains data on airbnb listings in the state of New York. It contains the following columns:

- `listing_id`: The unique identifier for a listing
- `description`: The description used on the listing
- `host_id`: Unique identifier for a host
- `host_name`: Name of host
- `neighbourhood_full`: Name of boroughs and neighbourhoods
- `coordinates`: Coordinates of listing (*latitude, longitude*)
- `Listing added`: Date of added listing
- `room_type`: Type of room
- `rating`: Rating from 0 to 5.
- `price`: Price per night for listing
- `number_of_reviews`: Amount of reviews received
- `last_review`: Date of last review
- `reviews_per_month`: Number of reviews per month
- `availability_365`: Number of days available per year
- `Number of stays`: Total number of stays thus far

## ✓ Getting started

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import missingno as msno
import datetime as dt
#to ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

```
# Read in the dataset
airbnb = pd.read_csv("airbnb.csv")
```

## Diagnosing data cleaning problems using simple **pandas** and visualizations

Some important and common methods needed to get a better understanding of DataFrames and diagnose potential data problems are the following:

- `.head()` prints the header of a DataFrame
- `.dtypes` prints datatypes of all columns in a DataFrame
- `.info()` provides a bird's eye view of column data types and missing values in a DataFrame
- `.describe()` returns a distribution of numeric columns in your DataFrame
- `.isna().sum()` allows us to break down the number of missing values per column in our DataFrame
- `.unique()` finds the number of unique values in a DataFrame column
- `sns.displot()` plots the distribution of one column in your DataFrame.

```
# Print the header of the DataFrame
airbnb.head()
```

| Unnamed: 0 | listing_id | name     | host_id                                           | host_name | neighbourhood_full |                            |
|------------|------------|----------|---------------------------------------------------|-----------|--------------------|----------------------------|
| 0          | 0          | 13740704 | Cozy,budget friendly, cable inc, private entra... | 20583125  | Michel             | Brooklyn, Flatlands        |
| 1          | 1          | 22005115 | Two floor apartment near Central Park             | 82746113  | Cecilia            | Manhattan, Upper West Side |
| 2          | 2          | 21667615 | Beautiful 1BR in Brooklyn Heights                 | 78251     | Leslie             | Brooklyn, Brooklyn Heights |
| 3          | 3          | 6425850  | Spacious, charming studio                         | 32715865  | Yelena             | Manhattan, Upper West Side |
| 4          | 4          | 22986519 | Bedroom on the lively Lower East Side             | 154262349 | Brooke             | Manhattan, Lower East Side |

We will notice that the `.head()` has already revealed a range of potential problems such as:

#### *Data type problems:*

- **Problem 1:** We can see that the `coordinates` column is probably a string (`str`) - most mapping functions require a latitude input, and longitude input, so it's best to split this column into two and convert the values to `float`.
- **Problem 2:** Similar to `coordinates` - the `price` column also is a string with `$` attached to each price point, we need to convert that to `float` if we want a good understanding of the dataset.
- **Problem 3:** We need to make sure date columns (`last_review` and `listing_added`) are in `datetime` to allow easier manipulation of the data.

#### *Missing data problems:*

- **Problem 4:** We can see that there are missing values in some columns, we'll get a better good look of that as we go along in the data cleaning process.

#### *Text/categorical data problems:*

- **Problem 5:** To be able to visualize number of listings by boroughs - we need to separate neighbourhood name from borough name in `neighbourhood_full` column.
- **Problem 6:** Looking at `room_type`, let's replace those values to make them `'Shared Room'`, `'Private Home/Apartment'`, `'Private Room'` and `'Hotel Room'`.

```
# Print data types of DataFrame
airbnb.dtypes
```

|                           | 0       |
|---------------------------|---------|
| <b>Unnamed: 0</b>         | int64   |
| <b>listing_id</b>         | int64   |
| <b>name</b>               | object  |
| <b>host_id</b>            | int64   |
| <b>host_name</b>          | object  |
| <b>neighbourhood_full</b> | object  |
| <b>coordinates</b>        | object  |
| <b>room_type</b>          | object  |
| <b>price</b>              | object  |
| <b>number_of_reviews</b>  | int64   |
| <b>last_review</b>        | object  |
| <b>reviews_per_month</b>  | float64 |
| <b>availability_365</b>   | int64   |
| <b>rating</b>             | float64 |
| <b>number_of_stays</b>    | float64 |
| <b>5_stars</b>            | float64 |
| <b>listing_added</b>      | object  |

**dtype:** object

Printing the data types confirms that `coordinates` and `price` need to be converted to `float`, and date columns need to be converted to `datetime` (*problems 1,2 3*)

```
# Print info of DataFrame
airbnb.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10019 entries, 0 to 10018
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            10019 non-null  int64
1   listing_id            10019 non-null  int64
2   name                  10014 non-null  object
3   host_id               10019 non-null  int64
4   host_name             10017 non-null  object
5   neighbourhood_full    10019 non-null  object
6   coordinates           10019 non-null  object
7   room_type             10019 non-null  object
8   price                 9781 non-null   object
9   number_of_reviews     10019 non-null  int64
10  last_review           7944 non-null   object
11  reviews_per_month    7944 non-null   float64
12  availability_365      10019 non-null  int64
13  rating                7944 non-null   float64
14  number_of_stays       7944 non-null   float64
15  5_stars               7944 non-null   float64
16  listing_added         10019 non-null  object
dtypes: float64(4), int64(5), object(8)
memory usage: 1.3+ MB

```

Printing the info confirms our guess about the following:

- There is missing data in the `price`, `last_review`, `reviews_per_month`, `rating`, `number_of_stays`, `5_stars` columns. It also seems that the missingness of `last_review`, `reviews_per_month`, `rating`, `number_of_stays`, `5_stars` are related since they have the same amount of missing data. We will confirm later with `missingno` (*problem 4*).

```

# Print number of missing values
airbnb.isna().sum()

```

|                    | 0    |
|--------------------|------|
| Unnamed: 0         | 0    |
| listing_id         | 0    |
| name               | 5    |
| host_id            | 0    |
| host_name          | 2    |
| neighbourhood_full | 0    |
| coordinates        | 0    |
| room_type          | 0    |
| price              | 238  |
| number_of_reviews  | 0    |
| last_review        | 2075 |
| reviews_per_month  | 2075 |
| availability_365   | 0    |
| rating             | 2075 |
| number_of_stays    | 2075 |
| 5_stars            | 2075 |
| listing_added      | 0    |

**dtype:** int64

As discussed during the lecture, there are a variety of ways of dealing with missing data that is dependent on type of missingness, as well as the business assumptions behind our data - our options could be:

- Dropping missing data (if the data dropped does not impact or skew our data)
- Imputing with statistical measures (median, mean, mode ...)
- Imputing with more complex algorithmic/machine learning based approaches
- Impute based on business assumptions of our data

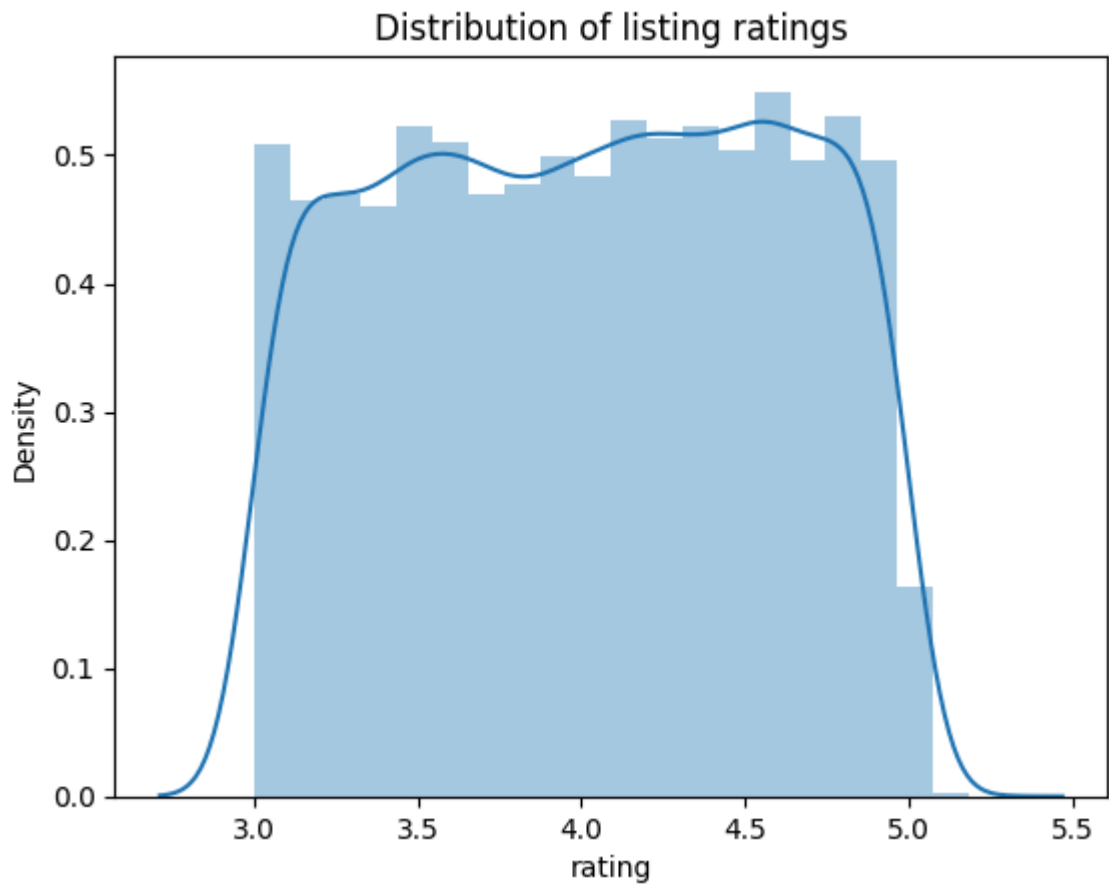
```
# Print description of DataFrame
airbnb.describe()
```

|              | Unnamed: 0   | listing_id   | host_id      | number_of_reviews | reviews_per_listing |
|--------------|--------------|--------------|--------------|-------------------|---------------------|
| <b>count</b> | 10019.000000 | 1.001900e+04 | 1.001900e+04 | 10019.000000      | 7944.000000         |
| <b>mean</b>  | 5009.000000  | 1.927634e+07 | 6.795923e+07 | 22.459727         | 1.927634e+07        |
| <b>std</b>   | 2892.380507  | 1.095056e+07 | 7.863106e+07 | 43.173896         | 1.095056e+07        |
| <b>min</b>   | 0.000000     | 3.831000e+03 | 2.787000e+03 | 0.000000          | 0.000000            |
| <b>25%</b>   | 2504.500000  | 9.674772e+06 | 7.910880e+06 | 1.000000          | 0.000000            |
| <b>50%</b>   | 5009.000000  | 2.007030e+07 | 3.165167e+07 | 5.000000          | 0.000000            |
| <b>75%</b>   | 7513.500000  | 2.933864e+07 | 1.074344e+08 | 22.000000         | 2.933864e+07        |
| <b>max</b>   | 10018.000000 | 3.648724e+07 | 2.741034e+08 | 510.000000        | 16.000000           |

- **Problem 7:** Looking at the maximum of the `rating` column - we see that it is out of range of `5` which is the maximum rating possible. We need to make sure we fix the range this column.

It's worth noting that `.describe()` does not offer a bird's eye view of all the out of range data we have, for example, what if we have date data in the future? Or given our dataset, `listing_added` dates that are in the future of `last_review` dates?

```
# Visualize the distribution of the rating column
sns.distplot(airbnb['rating'], bins = 20)
plt.title('Distribution of listing ratings')
plt.show()
```



```
# Find number of unique values in room_type column  
airbnb['room_type'].unique()
```

```
array(['Private room', 'Entire home/apt', 'Private', 'Shared room',  
      'PRIVATE ROOM', 'home', ' Shared room'], dtype=object)
```

- **Problem 8:** There are trailing spaces and capitalization issues with `room_type`, we need to fix this problem.

```
# How many values of different room_types do we have?  
airbnb['room_type'].value_counts()
```



|                 | count |
|-----------------|-------|
| room_type       |       |
| Entire home/apt | 5120  |
| Private room    | 4487  |
| Shared room     | 155   |
| Private         | 89    |
| Shared room     | 71    |
| home            | 66    |
| PRIVATE ROOM    | 31    |

dtype: int64

## Our to do list:

*Data type problems:*

- **Task 1:** Split `coordinates` into 2 columns and convert them to `float`
- **Task 2:** Remove `$` from `price` and convert it to `float`
- **Task 3:** Convert `listing_added` and `last_review` to `datetime`

*Text/categorical data problems:*

- **Task 4:** We need to collapse `room_type` into correct categories
- **Task 5:** Divide `neighbourhood_full` into 2 columns and making sure they are clean

*Data range problems:*

- **Task 6:** Make sure we set the correct maximum for `rating` column out of range values

*Dealing with missing data:*

- **Task 7:** Understand the type of missingness, and deal with the missing data in most of the remaining columns.

*Is that all though?*

- We need to investigate if we have duplicates in our data

- We need to make sure that data makes sense by applying some sanity checks on our DataFrame

## Q&A

- ✓ **Cleaning data**
- ✓ Data type problems

```
# Reminder of the DataFrame  
airbnb.head()
```

|   | Unnamed: 0 | listing_id | name                                              | host_id   | host_name | neighbourhood_full         |
|---|------------|------------|---------------------------------------------------|-----------|-----------|----------------------------|
| 0 | 0          | 13740704   | Cozy,budget friendly, cable inc, private entra... | 20583125  | Michel    | Brooklyn, Flatlands        |
| 1 | 1          | 22005115   | Two floor apartment near Central Park             | 82746113  | Cecilia   | Manhattan, Upper West Side |
| 2 | 2          | 21667615   | Beautiful 1BR in Brooklyn Heights                 | 78251     | Leslie    | Brooklyn, Brooklyn Heights |
| 3 | 3          | 6425850    | Spacious, charming studio                         | 32715865  | Yelena    | Manhattan, Upper West Side |
| 4 | 4          | 22986519   | Bedroom on the lively Lower East Side             | 154262349 | Brooke    | Manhattan, Lower East Side |

- ✓ **Task 1:** Replace `coordinates` with `latitude` and `longitude` columns

To perform this task, we will use the following methods:

- `.str.replace("", "")` replaces one string in each row of a column with another

- `.str.split("")` takes in a string and lets you split a column into two based on that string
- `.astype()` lets you convert a column from one type to another

```
# Remove "(" and ")" from coordinates
airbnb['coordinates'] = airbnb['coordinates'].str.replace("(", "")
airbnb['coordinates'] = airbnb['coordinates'].str.replace(")", "")
# Print the header of the column
airbnb['coordinates'].head()
```

|   | coordinates         |
|---|---------------------|
| 0 | 40.63222, -73.93398 |
| 1 | 40.78761, -73.96862 |
| 2 | 40.7007, -73.99517  |
| 3 | 40.79169, -73.97498 |
| 4 | 40.71884, -73.98354 |

**dtype:** object

```
# Split column into two
lat_long = airbnb['coordinates'].str.split(",", expand = True)
lat_long.head()
```

|   | 0        | 1         |
|---|----------|-----------|
| 0 | 40.63222 | -73.93398 |
| 1 | 40.78761 | -73.96862 |
| 2 | 40.7007  | -73.99517 |
| 3 | 40.79169 | -73.97498 |
| 4 | 40.71884 | -73.98354 |

```
# Assign correct columns to latitude and longitude columns in airbnb
airbnb['latitude'] = lat_long[0]
airbnb['longitude'] = lat_long[1]
# Print the header and confirm new column creation
airbnb.head()
```

| Unnamed: 0 | listing_id | name     | host_id                                           | host_name | neighbourhood_full |                            |
|------------|------------|----------|---------------------------------------------------|-----------|--------------------|----------------------------|
| 0          | 0          | 13740704 | Cozy,budget friendly, cable inc, private entra... | 20583125  | Michel             | Brooklyn, Flatlands        |
| 1          | 1          | 22005115 | Two floor apartment near Central Park             | 82746113  | Cecilia            | Manhattan, Upper West Side |
| 2          | 2          | 21667615 | Beautiful 1BR in Brooklyn Heights                 | 78251     | Leslie             | Brooklyn, Brooklyn Heights |
| 3          | 3          | 6425850  | Spacious, charming studio                         | 32715865  | Yelena             | Manhattan, Upper West Side |
| 4          | 4          | 22986519 | Bedroom on the lively Lower East Side             | 154262349 | Brooke             | Manhattan, Lower East Side |

```
# Print out dtypes again
airbnb.dtypes
```

|                           | 0       |
|---------------------------|---------|
| <b>Unnamed: 0</b>         | int64   |
| <b>listing_id</b>         | int64   |
| <b>name</b>               | object  |
| <b>host_id</b>            | int64   |
| <b>host_name</b>          | object  |
| <b>neighbourhood_full</b> | object  |
| <b>coordinates</b>        | object  |
| <b>room_type</b>          | object  |
| <b>price</b>              | object  |
| <b>number_of_reviews</b>  | int64   |
| <b>last_review</b>        | object  |
| <b>reviews_per_month</b>  | float64 |
| <b>availability_365</b>   | int64   |
| <b>rating</b>             | float64 |
| <b>number_of_stays</b>    | float64 |
| <b>5_stars</b>            | float64 |
| <b>listing_added</b>      | object  |
| <b>latitude</b>           | object  |
| <b>longitude</b>          | object  |

**dtype:** object

```
# Convert latitude and longitude to float
airbnb['latitude'] = airbnb['latitude'].astype('float')
airbnb['longitude'] = airbnb['longitude'].astype('float')
# Print dtypes again
airbnb.dtypes
```

|                    | 0       |
|--------------------|---------|
| Unnamed: 0         | int64   |
| listing_id         | int64   |
| name               | object  |
| host_id            | int64   |
| host_name          | object  |
| neighbourhood_full | object  |
| coordinates        | object  |
| room_type          | object  |
| price              | object  |
| number_of_reviews  | int64   |
| last_review        | object  |
| reviews_per_month  | float64 |
| availability_365   | int64   |
| rating             | float64 |
| number_of_stays    | float64 |
| 5_stars            | float64 |
| listing_added      | object  |
| latitude           | float64 |
| longitude          | float64 |

**dtype:** object

```
# Drop coordinates column
airbnb.drop('coordinates', axis = 1, inplace = True)
```

## ✓ Task 2: Remove \$ from price and convert it to float

To perform this task, we will be using the following methods:

- `.str.strip()` which removes a specified string from each row in a column
- `.astype()`

```
# Calculate mean of price without conversion
airbnb['price'].mean()
```

```
# Remove $ from price before conversion to float
airbnb['price'] = airbnb['price'].str.strip("$")
# Print header to make sure change was done
airbnb['price'].head()
```

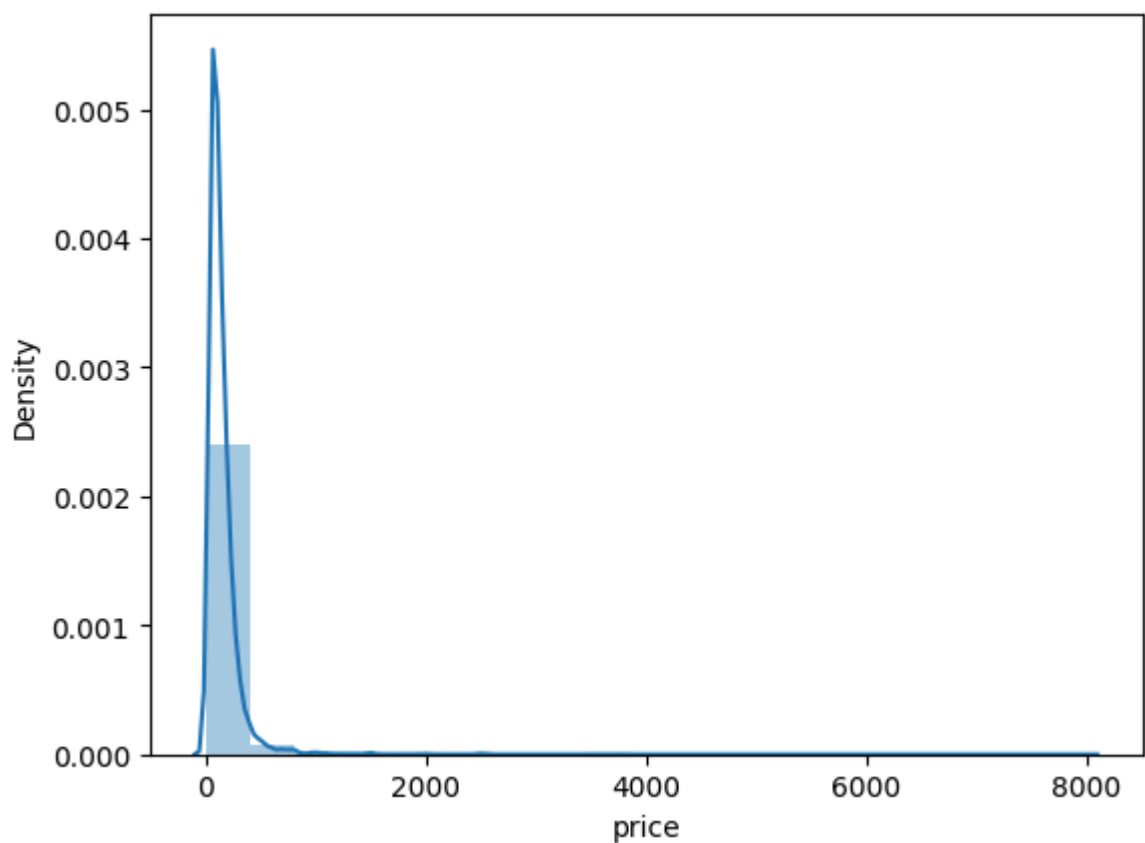
|   | price |
|---|-------|
| 0 | 45    |
| 1 | 135   |
| 2 | 150   |
| 3 | 86    |
| 4 | 160   |

**dtype:** object

```
# Convert price to float
airbnb['price'] = airbnb['price'].astype('float')
# Calculate mean of price after conversion
airbnb['price'].mean()
```

150.90512217564665

```
# Visualize distribution of prices
sns.distplot(airbnb['price'], bins = 20)
plt.show()
```



### Task 3: Convert `listing_added` and `last_review` columns to `datetime`

To perform this task, we will use the following functions:

- `pd.to_datetime(format = "")`
  - `format` takes in the desired date format `"%Y-%m-%d"`

```
# Print header of two columns
airbnb[['listing_added', 'last_review']].head()
```

|   | listing_added | last_review |
|---|---------------|-------------|
| 0 | 2018-06-08    | 2018-12-12  |
| 1 | 2018-12-25    | 2019-06-30  |
| 2 | 2018-08-15    | NaN         |
| 3 | 2017-03-20    | 2017-09-23  |
| 4 | 2020-10-23    | 2019-06-12  |

```
# Convert both columns to datetime
airbnb['listing_added'] = pd.to_datetime(airbnb['listing_added'], format = '
airbnb['last_review'] = pd.to_datetime(airbnb['last_review'], format = '%Y-%
```

```
# Print header and datatypes of both columns again
print(airbnb[['listing_added', 'last_review']].head())
print(airbnb[['listing_added', 'last_review']].dtypes)
```

|   | listing_added | last_review |
|---|---------------|-------------|
| 0 | 2018-06-08    | 2018-12-12  |
| 1 | 2018-12-25    | 2019-06-30  |
| 2 | 2018-08-15    | NaT         |
| 3 | 2017-03-20    | 2017-09-23  |
| 4 | 2020-10-23    | 2019-06-12  |

```
listing_added    datetime64[ns]
last_review      datetime64[ns]
dtype: object
```

### Text and categorical data problems

### Task 4: We need to collapse `room_type` into correct categories

To perform this task, we will be using the following methods:

- `.str.lower()` to lowercase all rows in a string column
- `.str.strip()` to remove all white spaces of each row in a string column
- `.replace()` to replace values in a column with another



```
# Print unique values of `room_type`
airbnb['room_type'].unique()
```

```
array(['Private room', 'Entire home/apt', 'Private', 'Shared room',
      'PRIVATE ROOM', 'home', ' Shared room'], dtype=object)
```

```
# Deal with capitalized values
airbnb['room_type'] = airbnb['room_type'].str.lower()
airbnb['room_type'].unique()
```

```
array(['private room', 'entire home/apt', 'private', 'shared room',
      'home', ' shared room'], dtype=object)
```

```
# Deal with trailing spaces
airbnb['room_type'] = airbnb['room_type'].str.strip()
airbnb['room_type'].unique()
```

```
array(['private room', 'entire home/apt', 'private', 'shared room',
      'home'], dtype=object)
```

```
# Replace values to 'Shared room', 'Entire place', 'Private room' and 'Hotel'
mappings = {'private room': 'Private Room',
            'private': 'Private Room',
            'entire home/apt': 'Entire place',
            'shared room': 'Shared room',
            'home': 'Entire place'}
```

```
# Replace values and collapse data
airbnb['room_type'] = airbnb['room_type'].replace(mappings)
airbnb['room_type'].unique()
```

```
array(['Private Room', 'Entire place', 'Shared room'], dtype=object)
```

✓ **Task 5:** Divide `neighbourhood_full` into 2 columns and making sure they are clean

```
# Print header of column
airbnb['neighbourhood_full'].head()
```

```
      neighbourhood_full
0      Brooklyn, Flatlands
1  Manhattan, Upper West Side
2  Brooklyn, Brooklyn Heights
3  Manhattan, Upper West Side
4  Manhattan, Lower East Side
```

```
dtype: object
```

```
# Split neighbourhood_full
borough_neighbourhood = airbnb['neighbourhood_full'].str.split(",", expand =
borough_neighbourhood.head()
```

|   | 0         | 1                |
|---|-----------|------------------|
| 0 | Brooklyn  | Flatlands        |
| 1 | Manhattan | Upper West Side  |
| 2 | Brooklyn  | Brooklyn Heights |
| 3 | Manhattan | Upper West Side  |
| 4 | Manhattan | Lower East Side  |

```
# Create borough and neighbourhood columns
airbnb['borough'] = borough_neighbourhood[0]
airbnb['neighbourhood'] = borough_neighbourhood[1]
# Print header of columns
airbnb[['neighbourhood_full', 'borough', 'neighbourhood']].head()
```

|   | neighbourhood_full         | borough   | neighbourhood    |
|---|----------------------------|-----------|------------------|
| 0 | Brooklyn, Flatlands        | Brooklyn  | Flatlands        |
| 1 | Manhattan, Upper West Side | Manhattan | Upper West Side  |
| 2 | Brooklyn, Brooklyn Heights | Brooklyn  | Brooklyn Heights |
| 3 | Manhattan, Upper West Side | Manhattan | Upper West Side  |
| 4 | Manhattan, Lower East Side | Manhattan | Lower East Side  |

```
# Drop neighbourhood_full column
airbnb.drop('neighbourhood_full', axis = 1, inplace = True)
```

```
# Print out unique values of borough and neighbourhood
print(airbnb['borough'].unique())
print(airbnb['neighbourhood'].unique())
```

```
['Brooklyn' 'Manhattan' 'Queens' 'Bronx' 'Staten Island']
[' Flatlands' ' Upper West Side' ' Brooklyn Heights' ' Lower East Side'
' Greenwich Village' ' Harlem' ' Sheepshead Bay' ' Theater District'
' Bushwick' ' Laurelton' ' Mott Haven' ' Flushing' ' Crown Heights'
' Midtown' ' Financial District' ' East Village' ' Park Slope'
' Washington Heights' ' Williamsburg' ' Chelsea' ' Bedford-Stuyvesant'
' Gowanus' ' Upper East Side' ' Ditmars Steinway' ' Cypress Hills'
" Hell's Kitchen" ' Ridgewood' ' Marble Hill' ' Kips Bay'
' Prospect Heights' ' East New York' ' Concord' ' Stapleton' ' Astoria'
' East Harlem' ' Sunnyside' ' Gramercy' ' Prospect-Lefferts Gardens'
' Sunset Park' ' Forest Hills' ' Windsor Terrace' ' Clinton Hill'
' Murray Hill' ' Flatiron District' ' Greenpoint' ' East Flatbush'
' Tribeca' ' Woodhaven' ' Fort Greene' ' Inwood' ' Chinatown'
' Rockaway Beach' ' Woodside' ' Bayside' ' Bensonhurst' ' SoHo'
' Red Hook' ' West Village' ' South Slope' ' Richmond Hill' ' Jamaica'
```

```
' Boerum Hill' ' St. George' ' Clason Point' ' College Point'
' Vinegar Hill' ' Rego Park' ' East Elmhurst' ' Kensington'
' Roosevelt Island' ' Ozone Park' ' Arverne' ' Fordham' ' Wakefield'
' Port Morris' ' Flatbush' ' Elmhurst' ' Bay Ridge' ' Canarsie'
' Little Italy' ' Civic Center' ' St. Albans' ' Morningside Heights'
' Morrisania' ' Two Bridges' ' Concourse' ' Downtown Brooklyn'
' Queens Village' ' Fieldston' ' Allerton' ' Grymes Hill'
' Carroll Gardens' ' Kingsbridge' ' Belle Harbor' ' NoHo' ' Maspeth'
' Borough Park' ' Midwood' ' Fresh Meadows' ' Battery Park City'
' Pelham Bay' ' Van Nest' ' Long Island City' ' Williamsbridge' ' Nolita'
' Kew Gardens' ' Corona' ' Middle Village' ' Jackson Heights'
' Gravesend' ' New Dorp Beach' ' Brighton Beach' ' Belmont'
' Eltingville' ' South Ozone Park' ' Stuyvesant Town' ' Far Rockaway'
' Longwood' ' Glendale' ' Dongan Hills' ' Hollis' ' Columbia St'
' Tompkinsville' ' Springfield Gardens' ' Concourse Village'
' Throgs Neck' ' Cobble Hill' ' South Beach' ' North Riverdale'
' Rosedale' ' Mount Hope' ' Edenwald' ' Parkchester' ' Mill Basin'
' Brownsville' ' Fort Hamilton' ' Howard Beach' ' Claremont Village'
' Pelham Gardens' ' Westchester Square' ' Kew Gardens Hills' ' Arrochar'
' Edgemere' ' Shore Acres' ' Dyker Heights' ' Mount Eden'
' Randall Manor' ' Jamaica Hills' ' Norwood' ' Tottenville'
' Cambria Heights' ' East Morrisania' ' Soundview' ' Morris Park'
' Bronxdale' ' Bellerose' ' Navy Yard' ' Unionport' ' DUMBO' ' Bayswater'
' Tremont' ' Highbridge' ' Graniteville' ' Baychester' ' Morris Heights'
' Coney Island' ' Briarwood' ' West Farms' "Prince's Bay"
' University Heights' ' Clifton' ' Bay Terrace' ' City Island'
' West Brighton' ' Douglaston' ' Schuylerville' ' Port Richmond'
' Oakwood' ' Westerleigh' ' Little Neck' ' Holliswood' ' Bergen Beach'
' Castle Hill' ' Olinville' ' Rosebank' ' Melrose' ' Whitestone'
' Grant City' ' Riverdale' ' Neponsit' ' Bath Beach' ' Mariners Harbor'
' Hunts Point' ' Great Kills' ' Midland Beach' ' Eastchester'
' Jamaica Estates' ' New Springville']
```

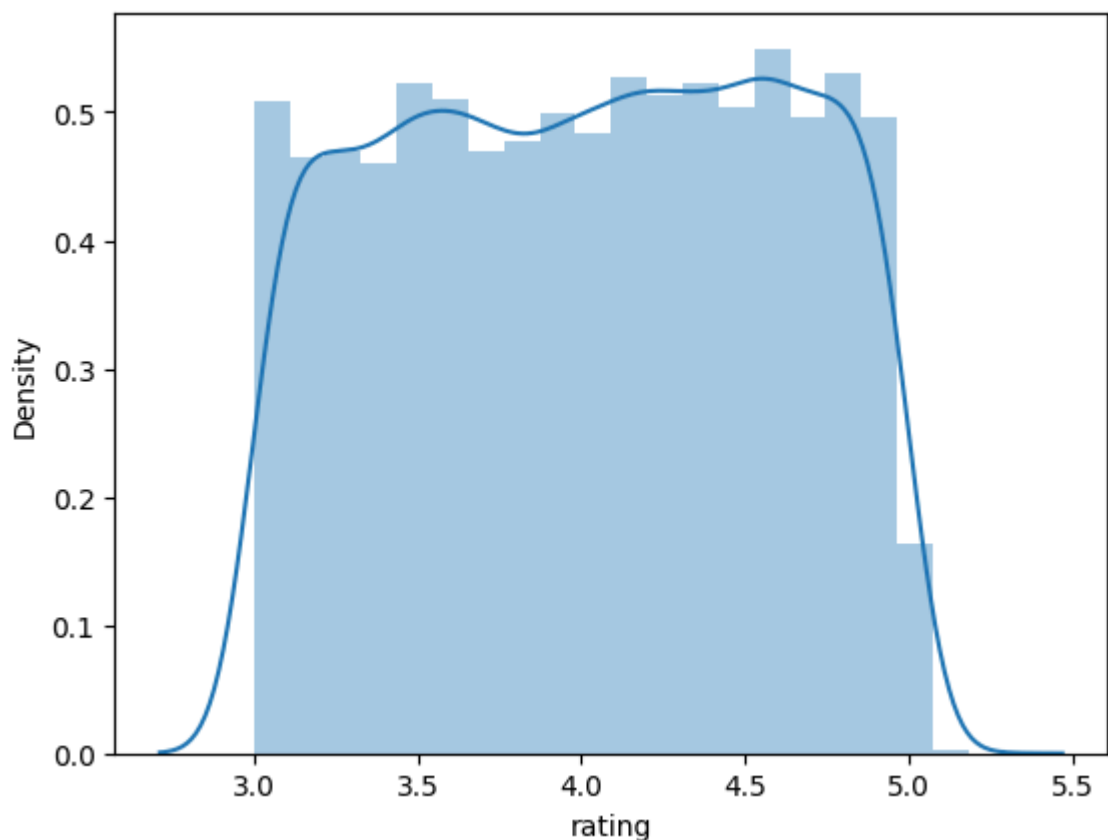
```
# Strip white space from neighbourhood column
airbnb['neighbourhood'] = airbnb['neighbourhood'].str.strip()
# Print unique values again
print(airbnb['neighbourhood'].unique())
```

```
['Flatlands' 'Upper West Side' 'Brooklyn Heights' 'Lower East Side'
'Greenwich Village' 'Harlem' 'Sheepshead Bay' 'Theater District'
'Bushwick' 'Laurelton' 'Mott Haven' 'Flushing' 'Crown Heights' 'Midtown'
'Financial District' 'East Village' 'Park Slope' 'Washington Heights'
'Williamsburg' 'Chelsea' 'Bedford-Stuyvesant' 'Gowanus' 'Upper East Side'
'Ditmars Steinway' 'Cypress Hills' "Hell's Kitchen" 'Ridgewood'
'Marble Hill' 'Kips Bay' 'Prospect Heights' 'East New York' 'Concord'
'Stapleton' 'Astoria' 'East Harlem' 'Sunnyside' 'Gramercy'
'Prospect-Lefferts Gardens' 'Sunset Park' 'Forest Hills'
'Windsor Terrace' 'Clinton Hill' 'Murray Hill' 'Flatiron District'
'Greenpoint' 'East Flatbush' 'Tribeca' 'Woodhaven' 'Fort Greene' 'Inwood'
'Chinatown' 'Rockaway Beach' 'Woodside' 'Bayside' 'Bensonhurst' 'SoHo'
'Red Hook' 'West Village' 'South Slope' 'Richmond Hill' 'Jamaica'
'Boerum Hill' 'St. George' 'Clason Point' 'College Point' 'Vinegar Hill'
'Rego Park' 'East Elmhurst' 'Kensington' 'Roosevelt Island' 'Ozone Park'
'Arverne' 'Fordham' 'Wakefield' 'Port Morris' 'Flatbush' 'Elmhurst'
'Bay Ridge' 'Canarsie' 'Little Italy' 'Civic Center' 'St. Albans'
'Morningside Heights' 'Morrisania' 'Two Bridges' 'Concourse'
'Downtown Brooklyn' 'Queens Village' 'Fieldston' 'Allerton' 'Grymes Hill'
'Carroll Gardens' 'Kingsbridge' 'Belle Harbor' 'NoHo' 'Maspeth'
'Borough Park' 'Midwood' 'Fresh Meadows' 'Battery Park City' 'Pelham Bay']
```

```
'Van Nest' 'Long Island City' 'Williamsbridge' 'Nolita' 'Kew Gardens'
'Corona' 'Middle Village' 'Jackson Heights' 'Gravesend' 'New Dorp Beach'
'Brighton Beach' 'Belmont' 'Eltingville' 'South Ozone Park'
'Stuyvesant Town' 'Far Rockaway' 'Longwood' 'Glendale' 'Dongan Hills'
'Hollis' 'Columbia St' 'Tompkinsville' 'Springfield Gardens'
'Concourse Village' 'Throgs Neck' 'Cobble Hill' 'South Beach'
'North Riverdale' 'Rosedale' 'Mount Hope' 'Edenwald' 'Parkchester'
'Mill Basin' 'Brownsville' 'Fort Hamilton' 'Howard Beach'
'Claremont Village' 'Pelham Gardens' 'Westchester Square'
'Kew Gardens Hills' 'Arrochar' 'Edgemere' 'Shore Acres' 'Dyker Heights'
'Mount Eden' 'Randall Manor' 'Jamaica Hills' 'Norwood' 'Tottenville'
'Cambria Heights' 'East Morrisania' 'Soundview' 'Morris Park' 'Bronxdale'
'Bellerose' 'Navy Yard' 'Unionport' 'DUMBO' 'Bayswater' 'Tremont'
'Highbridge' 'Graniteville' 'Baychester' 'Morris Heights' 'Coney Island'
'Briarwood' 'West Farms' "Prince's Bay" 'University Heights' 'Clifton'
'Bay Terrace' 'City Island' 'West Brighton' 'Douglaston' 'Schuylerville'
'Port Richmond' 'Oakwood' 'Westerleigh' 'Little Neck' 'Holliswood'
'Bergen Beach' 'Castle Hill' 'Olinville' 'Rosebank' 'Melrose'
'Whitestone' 'Grant City' 'Riverdale' 'Neponsit' 'Bath Beach'
'Mariners Harbor' 'Hunts Point' 'Great Kills' 'Midland Beach'
'Eastchester' 'Jamaica Estates' 'New Springville']
```

✓ **Task 6:** Make sure we set the correct maximum for `rating` column out of range values

```
# Visualize the rating column again
sns.distplot(airbnb['rating'], bins = 20)
plt.show()
```

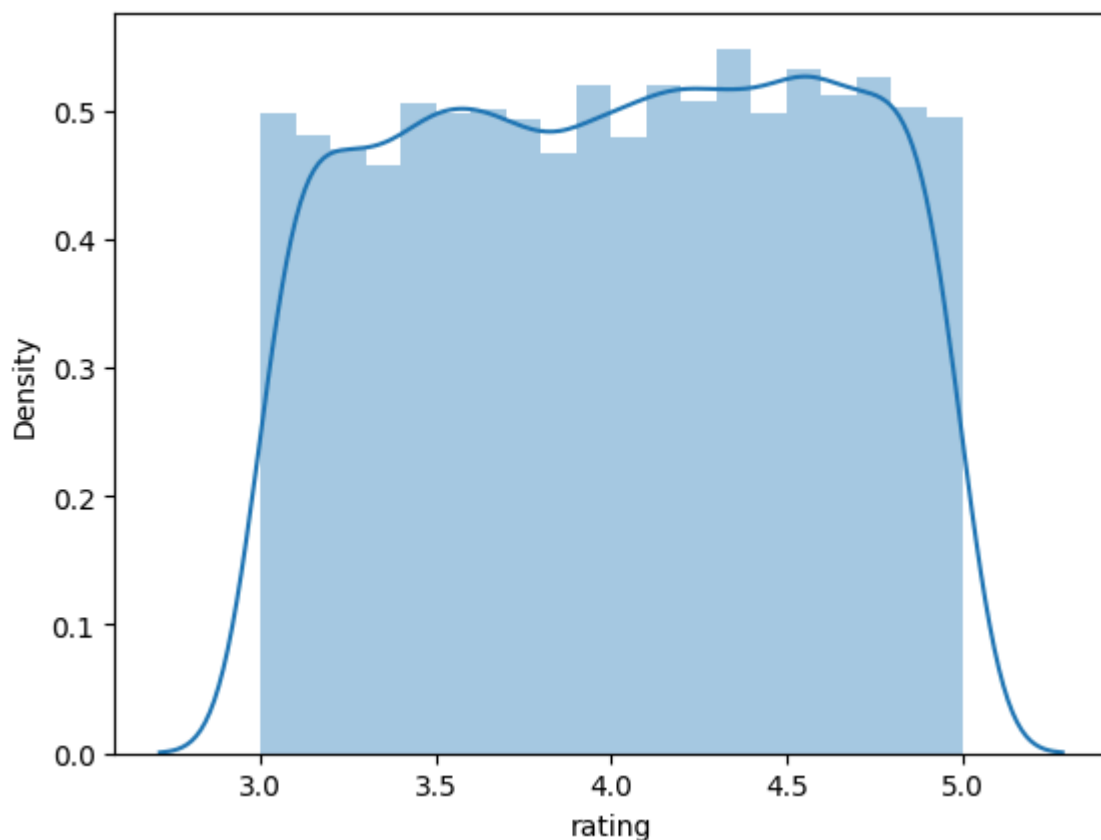


```
# Isolate rows of rating > 5.0
airbnb[airbnb['rating'] > 5.0]
```

|      | Unnamed:<br>0 | listing_id | name                                                   | host_id   | host_name | room_type       | price |
|------|---------------|------------|--------------------------------------------------------|-----------|-----------|-----------------|-------|
| 1925 | 1925          | 36411407   | Brand new<br>1 bedroom<br>steps from<br>Soho!          | 33917435  | Mike      | Entire<br>place | 150.  |
| 6732 | 6732          | 36390226   | Comfortable<br>clean<br>Bedstuy<br>private<br>room     | 267932490 | Angela    | Private<br>Room | 45.   |
| 8821 | 8821          | 36413632   | Spacious<br>2BR in<br>Beautiful<br>Brooklyn<br>Heights | 6608220   | Matt      | Entire<br>place | 550.  |

```
# Drop these rows and make sure we have effected changes
airbnb.drop(airbnb[airbnb['rating'] > 5.0].index, inplace = True)
```

```
# Visualize the rating column again
sns.distplot(airbnb['rating'], bins = 20)
plt.show()
```



```
# Get the maximum
airbnb['rating'].max()
```

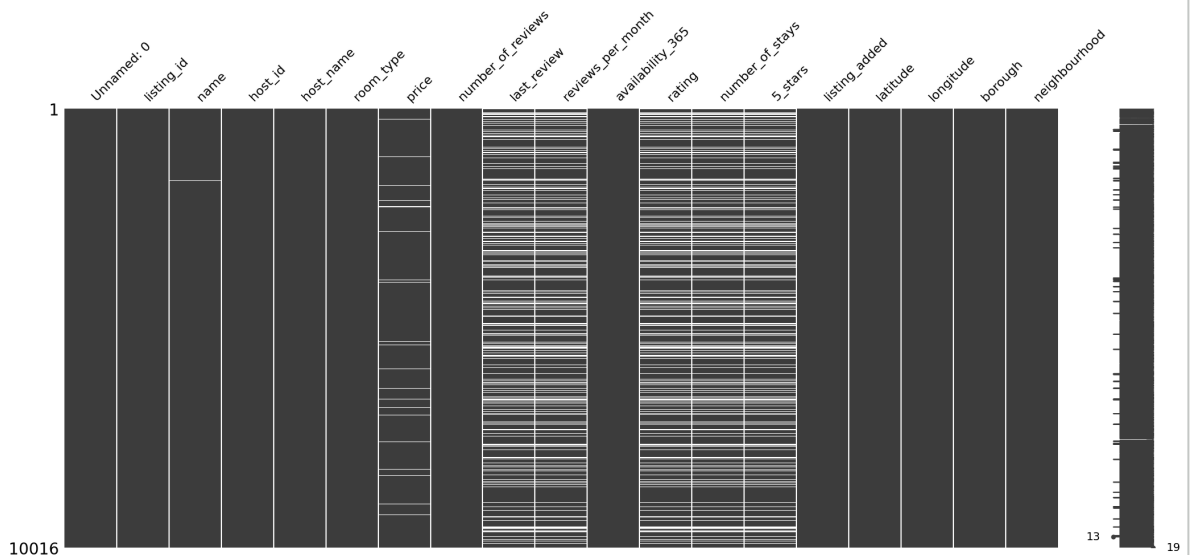
```
4.999561104773598
```

## ✓ Dealing with missing data

The `missingno` (imported as `msno`) package is great for visualizing missing data - we will be using:

- `msno.matrix()` visualizes a missingness matrix
- `msno.bar()` visualizes a missingness barplot
- `plt.show()` to show the plot

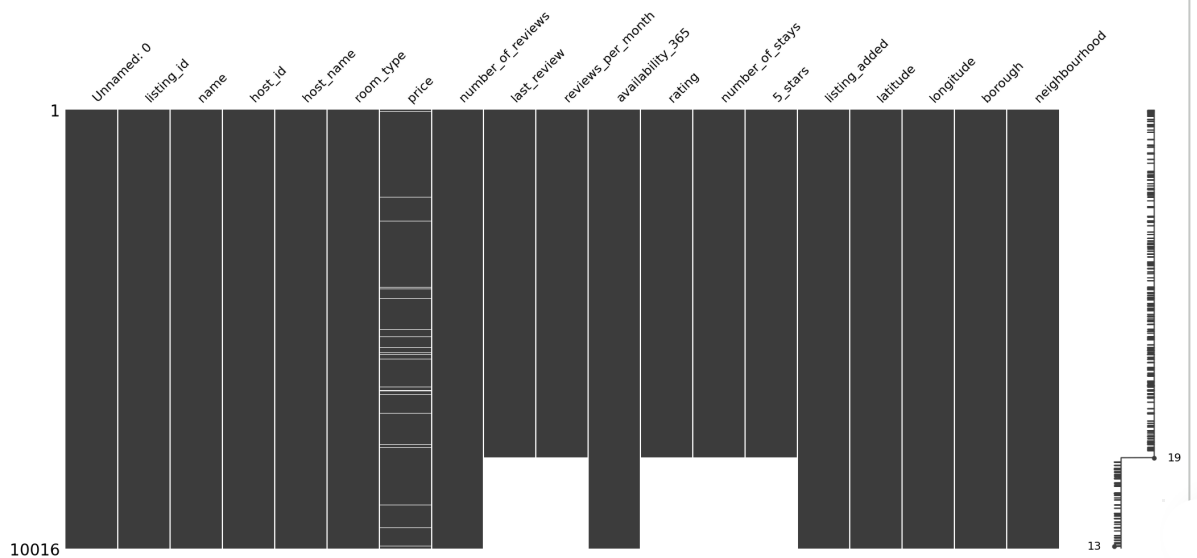
```
# Visualize the missingness
msno.matrix(airbnb)
plt.show()
```



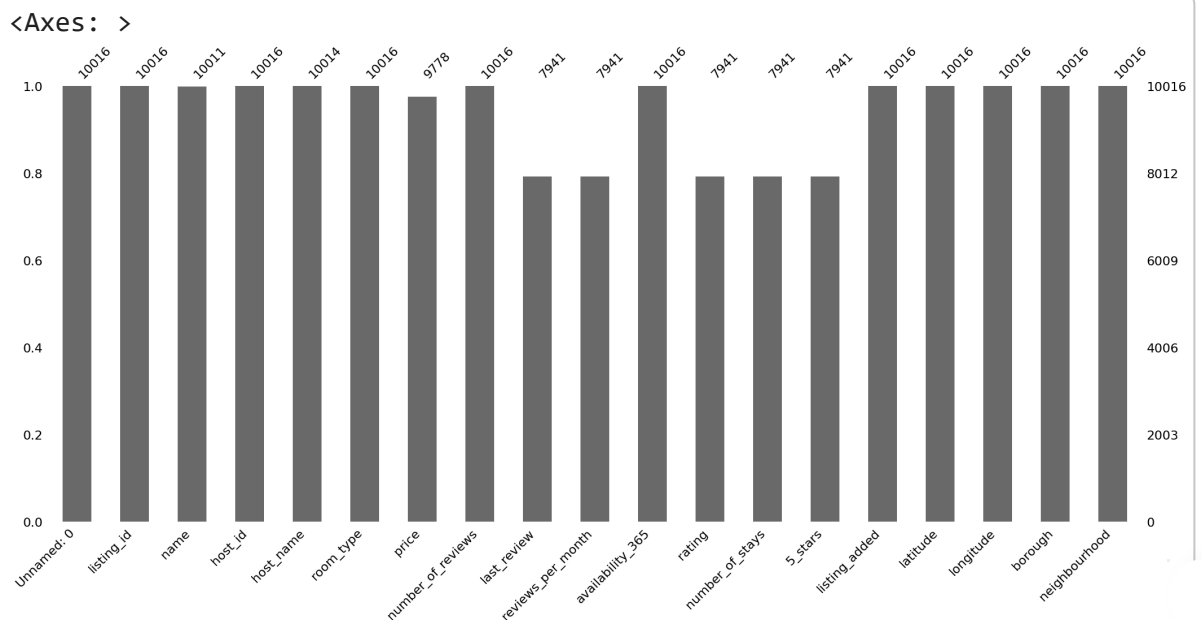
Looking at the missingness matrix, we can see that missing values are almost identical between `last_review`, `reviews_per_month`, `rating`, `number_of_stays`, and `5_stars`. Let's confirm this further by sorting on `rating`.

```
# Visualize the missingness on sorted values
msno.matrix(airbnb.sort_values(by = 'rating'))
```

```
plt.show()
```



```
# Missingness barplot  
msno.bar(airbnb)
```



Treating the `rating`, `number_of_stays`, `5_stars`, `reviews_per_month` columns

```
# Understand DataFrame with missing values in rating, number_of_stays, 5_stars
airbnb[airbnb['rating'].isna()].describe()
```

|       | Unnamed: 0   | listing_id   | host_id      | price       | number_of_reviews |
|-------|--------------|--------------|--------------|-------------|-------------------|
| count | 2075.000000  | 2.075000e+03 | 2.075000e+03 | 2028.000000 | 2075.000000       |
| mean  | 4983.181205  | 2.274238e+07 | 8.022455e+07 | 191.553748  | 0.000000          |
| min   | 2.000000     | 6.358800e+04 | 1.475100e+04 | 0.000000    | 0.000000          |
| 25%   | 2461.500000  | 1.232923e+07 | 1.224305e+07 | 70.000000   | 0.000000          |
| 50%   | 4987.000000  | 2.345182e+07 | 4.040116e+07 | 120.000000  | 0.000000          |
| 75%   | 7390.000000  | 3.400364e+07 | 1.333498e+08 | 205.250000  | 0.000000          |
| max   | 10007.000000 | 3.648724e+07 | 2.741034e+08 | 5250.000000 | 0.000000          |



```
# Understand DataFrame with missing values in rating, number_of_stays, 5_stars
airbnb[~airbnb['rating'].isna()].describe()
```

|       | Unnamed: 0   | listing_id   | host_id      | price       | number_of_reviews |
|-------|--------------|--------------|--------------|-------------|-------------------|
| count | 7941.000000  | 7.941000e+03 | 7.941000e+03 | 7750.000000 | 7941.000000       |
| mean  | 5015.437854  | 1.836419e+07 | 6.474110e+07 | 140.230581  | 28.336483         |
| min   | 0.000000     | 3.831000e+03 | 2.787000e+03 | 0.000000    | 1.000000          |
| 25%   | 2515.000000  | 8.964308e+06 | 7.138163e+06 | 69.000000   | 3.000000          |
| 50%   | 5015.000000  | 1.927436e+07 | 2.947846e+07 | 105.000000  | 9.000000          |
| 75%   | 7544.000000  | 2.787660e+07 | 1.016578e+08 | 170.000000  | 32.000000         |
| max   | 10018.000000 | 3.635113e+07 | 2.733615e+08 | 8000.000000 | 510.000000        |

Looking at the missing data in the DataFrame - we can see that `number_of_reviews` across all missing rows is 0. We can infer that these listings have never been visited - hence could be inferred they're inactive/have never been visited.

We can impute them as following:

- Set `NaN` for `reviews_per_month`, `number_of_stays`, `5_stars` to 0.
- Since a `rating` did not happen, let's keep the column as is - but create a new column named `rated` that takes in `1` if yes, `0` if no.
- We will also leave `last_review` as is.

```
# Impute missing data
airbnb = airbnb.fillna({'reviews_per_month':0,
                       'number_of_stays':0,
                       '5_stars':0})

# Create is_rated column
is_rated = np.where(airbnb['rating'].isna() == True, 0, 1)
airbnb['is_rated'] = is_rated
```

## Treating the `price` column

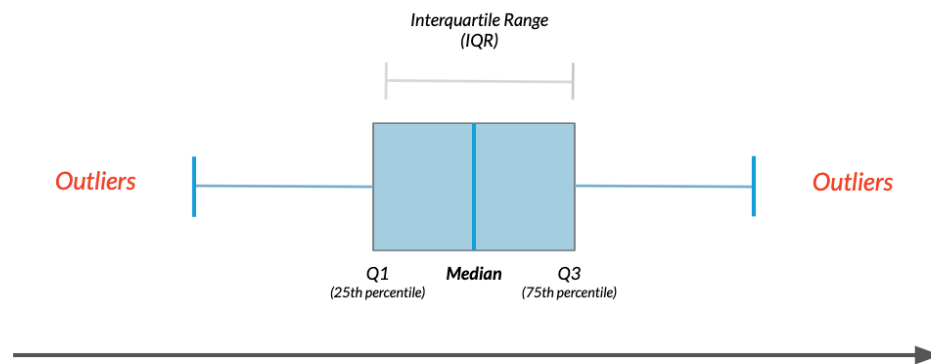
```
# Investigate DataFrame with missing values in price
airbnb[airbnb['price'].isna()].describe()
```

|              | Unnamed: 0   | listing_id   | host_id      | price | number_of_reviews |
|--------------|--------------|--------------|--------------|-------|-------------------|
| <b>count</b> | 238.000000   | 2.380000e+02 | 2.380000e+02 | 0.0   | 238.000000        |
| <b>mean</b>  | 5221.634454  | 1.805656e+07 | 6.262965e+07 | NaN   | 22.445378         |
| <b>min</b>   | 172.000000   | 8.092400e+04 | 1.145900e+05 | NaN   | 0.000000          |
| <b>25%</b>   | 2553.250000  | 8.282298e+06 | 6.034050e+06 | NaN   | 1.000000          |
| <b>50%</b>   | 5296.000000  | 1.863600e+07 | 2.809524e+07 | NaN   | 6.000000          |
| <b>75%</b>   | 7802.500000  | 2.753759e+07 | 1.009006e+08 | NaN   | 26.000000         |
| <b>max</b>   | 10010.000000 | 3.638875e+07 | 2.668265e+08 | NaN   | 207.000000        |

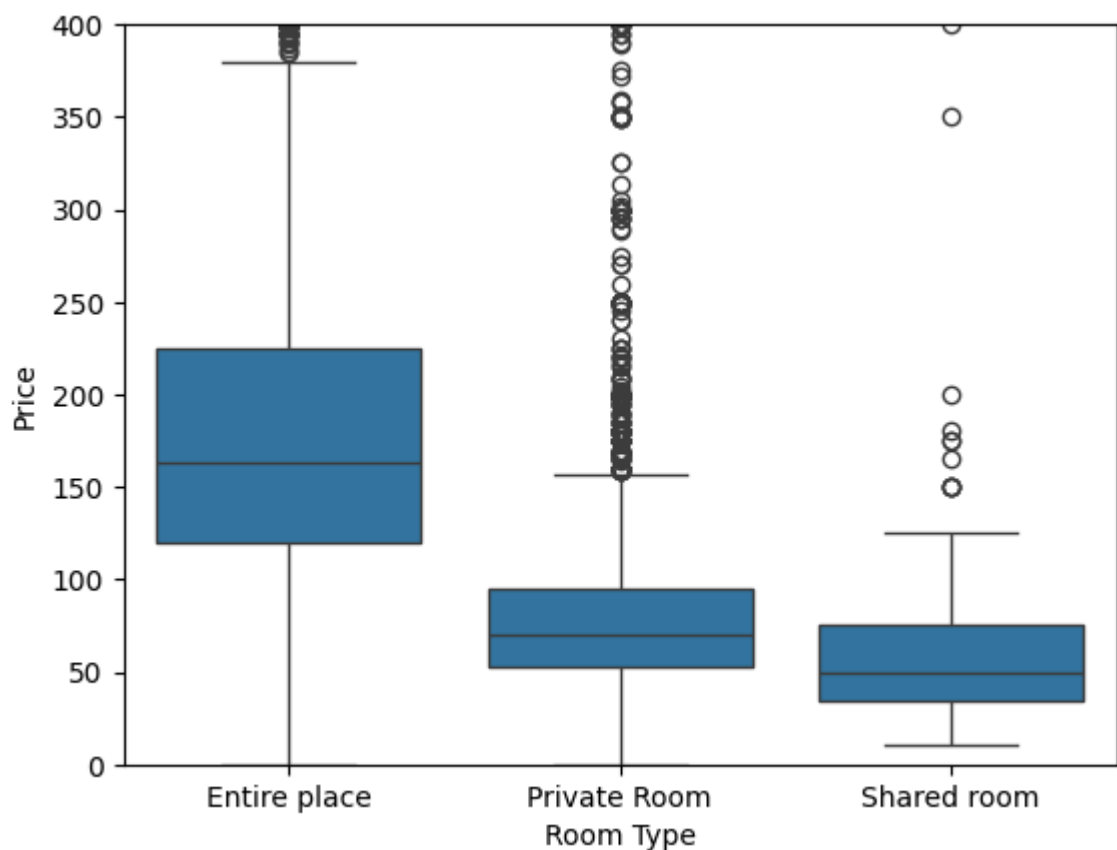
```
# Investigate DataFrame with missing values in price
airbnb[~airbnb['price'].isna()].describe()
```

|              | Unnamed: 0   | listing_id   | host_id      | price       | number_of_reviews |
|--------------|--------------|--------------|--------------|-------------|-------------------|
| <b>count</b> | 9778.000000  | 9.778000e+03 | 9.778000e+03 | 9778.000000 | 9778.000000       |
| <b>mean</b>  | 5003.573737  | 1.930078e+07 | 6.807826e+07 | 150.87523   | 22.466558         |
| <b>min</b>   | 0.000000     | 3.831000e+03 | 2.787000e+03 | 0.000000    | 0.000000          |
| <b>25%</b>   | 2504.250000  | 9.696668e+06 | 7.952219e+06 | 69.000000   | 1.000000          |
| <b>50%</b>   | 5000.500000  | 2.012299e+07 | 3.167018e+07 | 106.000000  | 5.000000          |
| <b>75%</b>   | 7502.750000  | 2.938694e+07 | 1.074344e+08 | 180.000000  | 22.000000         |
| <b>max</b>   | 10018.000000 | 3.648724e+07 | 2.741034e+08 | 8000.000000 | 510.000000        |

From a common sense perspective, the most predictive factor for a room's price is the `room_type` column, so let's visualize how price varies by room type with `sns.boxplot()` which displays the following information:



```
# Visualize relationship between price and room_type
sns.boxplot(x = 'room_type', y = 'price', data = airbnb)
plt.ylim(0, 400)
plt.xlabel('Room Type')
plt.ylabel('Price')
plt.show()
```



```
# Impute price based on conditions
airbnb.loc[(airbnb['price'].isna()) & (airbnb['room_type'] == 'Entire place')]
airbnb.loc[(airbnb['price'].isna()) & (airbnb['room_type'] == 'Private Room')]
airbnb.loc[(airbnb['price'].isna()) & (airbnb['room_type'] == 'Shared Room')]
```

```
# Confirm price has been imputed
airbnb.isna().sum()
```

|                   | 0    |
|-------------------|------|
| listing_id        | 0    |
| 5_stars           | 0    |
| Unnamed: 0        | 0    |
| availability_365  | 0    |
| borough           | 0    |
| host_id           | 0    |
| host_name         | 2    |
| is Rated          | 0    |
| last_review       | 2071 |
| latitude          | 0    |
| longitude         | 0    |
| name              | 5    |
| neighbourhood     | 0    |
| number_of_reviews | 0    |
| number_of_stays   | 0    |
| reviews_per_month | 0    |
| room_type         | 0    |
| price             | 7    |
| rating            | 2071 |
| listing_added     | 0    |

dtype: int64

## What's still to be done?

Albeit we've done a significant amount of data cleaning tasks, there are still a couple of problems we have yet to diagnose. When cleaning data, we need to consider:

- Values that do not make any sense (for example: are there values of `Last_review` that older than `Listing_added`? Are there listings in the future?)
- Presence of duplicates values - and how to deal with them?

## Task 8: Do we have consistent date data?

```
# Doing some sanity checks on date data
today = dt.date.today()
```

```
# Are there reviews in the future?
airbnb[airbnb['last_review'].dt.date > today]
```

```
Unnamed: 0  listing_id  name  host_id  host_name  room_type  price  number_of_
```

---

```
# Are there listings in the future?
airbnb[airbnb['listing_added'].dt.date > today]
```

```
Unnamed: 0  listing_id  name  host_id  host_name  room_type  price  number_o
```

---

```
# Drop these rows since they are only 4 rows
airbnb = airbnb[~(airbnb['listing_added'].dt.date > today)]
```

```
# Are there any listings with listing_added > last_review
inconsistent_dates = airbnb[airbnb['listing_added'].dt.date > airbnb['last_r
inconsistent_dates
```

|     | Unnamed: 0 | listing_id | name                                              | host_id   | host_name | room_type    | price |
|-----|------------|------------|---------------------------------------------------|-----------|-----------|--------------|-------|
| 4   | 4          | 22986519   | Bedroom on the lively Lower East Side             | 154262349 | Brooke    | Private Room | 160.0 |
| 50  | 50         | 20783900   | Marvelous Manhattan Marble Hill Private Suites    | 148960265 | Randy     | Private Room | 93.0  |
| 60  | 60         | 1908852    | Oversized Studio By Columbus Circle               | 684629    | Alana     | Entire place | 189.0 |
| 124 | 124        | 28659894   | Private bedroom in prime Bushwick! Near Trains!!! | 216235179 | Nina      | Private Room | 55.0  |
| 511 | 511        | 33619855   | Modern & Spacious in trendy Crown Heights         | 253354074 | Yehudis   | Entire place | 150.0 |
| 521 | 521        | 25317793   | Awesome Cozy Room in The Heart of Sunnyside!      | 136406167 | Kara      | Private Room | 65.0  |

```
# Drop these rows since they are only 2 rows
airbnb.drop(inconsistent_dates.index, inplace = True)
```

## ✓ Task 9: Let's deal with duplicate data

There are two notable types of duplicate data:

- Identical duplicate data across all columns
- Identical duplicate data cross most or some columns

To diagnose, and deal with duplicate data, we will be using the following methods and functions:

- `.duplicated(subset = , keep = )`

- `subset` lets us pick one or more columns with duplicate values.
- `keep` returns lets us return all instances of duplicate values.
- `.drop_duplicates(subset = , keep = )`

```
# Print the header of the DataFrame again
airbnb.head()
```

|   | Unnamed: 0 | listing_id | name                                              | host_id  | host_name | room_type    | price | n |
|---|------------|------------|---------------------------------------------------|----------|-----------|--------------|-------|---|
| 0 | 0          | 13740704   | Cozy,budget friendly, cable inc, private entra... | 20583125 | Michel    | Private Room | 45.0  |   |
| 1 | 1          | 22005115   | Two floor apartment near Central Park             | 82746113 | Cecilia   | Entire place | 135.0 |   |
| 2 | 2          | 21667615   | Beautiful 1BR in Brooklyn Heights                 | 78251    | Leslie    | Entire place | 150.0 |   |
| 3 | 3          | 6425850    | Spacious, charming studio                         | 32715865 | Yelena    | Entire place | 86.0  |   |
| 5 | 5          | 271954     | Beautiful brownstone apartment                    | 1423798  | Aj        | Entire place | 150.0 |   |

```
# Find duplicates
duplicates = airbnb.duplicated(subset = 'listing_id', keep = False)
print(duplicates)
```

```
0      False
1      False
2      False
3      False
5      False
...
10014   False
10015   False
10016   False
10017   False
10018   False
Length: 10010, dtype: bool
```

```
# Find duplicates
airbnb[duplicates].sort_values('listing_id')
```





|      | Unnamed: 0 | listing_id | name                                              | host_id  | host_name           | room_t       |
|------|------------|------------|---------------------------------------------------|----------|---------------------|--------------|
| 1145 | 1145       | 253806     | Loft Suite @ The Box House Hotel                  | 417504   | The Box House Hotel | Entire place |
| 6562 | 6562       | 253806     | Loft Suite @ The Box House Hotel                  | 417504   | The Box House Hotel | Entire place |
| 8699 | 8699       | 2044392    | The heart of Williamsburg 2 bedroom               | 620218   | Sarah               | Entire place |
| 5761 | 5761       | 2044392    | The heart of Williamsburg 2 bedroom               | 620218   | Sarah               | Entire place |
| 4187 | 4187       | 4244242    | Best Bedroom in Bedstuy/Bushwick. Ensuite bath... | 22023014 | BrooklynSleeps      | Full Room    |
| 2871 | 2871       | 4244242    | Best Bedroom in Bedstuy/Bushwick. Ensuite bath... | 22023014 | BrooklynSleeps      | Private Room |
| 77   | 77         | 7319856    | 450ft Square Studio in Gramercy NY                | 11773680 | Adam                | Entire place |
| 2255 | 2255       | 7319856    | 450ft Square Studio in Gramercy NY                | 11773680 | Adam                | Entire place |
| 555  | 555        | 9078222    | Prospect Park 3 bdrm, Sleeps 8 (#2)               | 47219962 | Babajide            | Entire place |
| 7933 | 7933       | 9078222    | Prospect Park 3 bdrm, Sleeps 8 (#2)               | 47219962 | Babajide            | Entire place |
| 1481 | 1481       | 15027024   | Newly renovated 1bd on lively & historic St Marks | 8344620  | Ethan               | Entire place |
| 3430 | 3430       | 15027024   | Newly renovated 1bd on lively & historic St Marks | 8344620  | Ethan               | Entire place |
| 3065 | 3065       | 16518377   | East Village 1BR Apt with all the amenities       | 3012457  | Cody                | Entire place |
| 5397 | 5397       | 16518377   | East Village 1BR Apt with all the amenities       | 3012457  | Cody                | Entire place |
| 3007 | 3007       | 17861841   | THE CREATIVE COZY ROOM                            | 47591528 | Janessa             | Private Room |
| 368  | 368        | 17861841   | THE CREATIVE COZY ROOM                            | 47591528 | Janessa             | Private Room |

|             |      |          |                                                   |          |          |           |
|-------------|------|----------|---------------------------------------------------|----------|----------|-----------|
| <b>6438</b> | 6438 | 19477677 | Huge sunny room next to subway!                   | 25038748 | Justin   | Pri<br>Re |
| <b>5251</b> | 5251 | 19477677 | Huge sunny room next to subway!                   | 25038748 | Justin   | Pri<br>Re |
| <b>5454</b> | 5454 | 21106251 | Private Bedroom in Great Brooklyn Apartment       | 25354313 | Tommy    | Pri<br>Re |
| <b>6832</b> | 6832 | 21106251 | Private Bedroom in Great Brooklyn Apartment       | 25354313 | Tommy    | Pri<br>Re |
| <b>5246</b> | 5246 | 22014840 | Sunny Bedroom Only 1 Metro Stop to Manhattan      | 32093643 | Scarlett | Pri<br>Re |
| <b>6068</b> | 6068 | 22014840 | Sunny Bedroom Only 1 Metro Stop to Manhattan      | 32093643 | Scarlett | Pri<br>Re |
| <b>5861</b> | 5861 | 23990868 | 1 Bedroom in Luxury Building                      | 4447548  | Grace    | El<br>p   |
| <b>6132</b> | 6132 | 23990868 | 1 Bedroom in Luxury Building                      | 4447548  | Grace    | El<br>p   |
| <b>7769</b> | 7769 | 26554879 | East Village/Union Square Flat                    | 17400431 | Bob      | El<br>p   |
| <b>7116</b> | 7116 | 26554879 | East Village/Union Square Flat                    | 17400431 | Bob      | El<br>p   |
| <b>5236</b> | 5236 | 29844951 | Cozy Home In Queens                               | 49946447 | Rah      | Pri<br>Re |
| <b>9425</b> | 9425 | 29844951 | Cozy Home In Queens                               | 49946447 | Rah      | Pri<br>Re |
| <b>7316</b> | 7316 | 31470004 | Private bedroom/Bathroom in a 2 bedroom apartment | 71241932 | Max      | Pri<br>Re |
| <b>9322</b> | 9322 | 31470004 | Private bedroom/Bathroom in a 2 bedroom apartment | 71241932 | Max      | Pri<br>Re |
| <b>1338</b> | 1338 | 32610834 | Manhattan by the water!                           | 12132369 | Omar     | El<br>p   |
| <b>6313</b> | 6313 | 32610834 | Manhattan by the water!                           | 12132369 | Omar     | El<br>p   |
| <b>6085</b> | 6085 | 33346762 | 2BR Apartment in Brownstone Brooklyn!             | 50321289 | Avery    | El<br>p   |
| <b>2118</b> | 2118 | 33346762 | 2BR Apartment in Brownstone Brooklyn!             | 50321289 | Avery    | El<br>p   |

```
# Remove identical duplicates
airbnb = airbnb.drop_duplicates()
```

Laundry

```
# Find non-identical duplicates
duplicates = airbnb.duplicated(subset = 'listing_id', keep = False)
```

Collected 1BR +

```
# Show all duplicates
airbnb[duplicates].sort_values('listing_id')
```

|      |      |          |                                           |           |        |          |
|------|------|----------|-------------------------------------------|-----------|--------|----------|
| 1030 | 1030 | 33040737 | Chelsea,<br>Manhattan                     | 117365574 | Maria  | R        |
| 3340 | 3340 | 35646737 | Private Cabins @<br>Chelsea,<br>Manhattan | 117365574 | Maria  | Pri<br>R |
| 7155 | 7155 | 35801208 | Comfy 2 bedroom<br>Close To<br>Manhattan  | 256911412 | Taylor | E<br>p   |
| 9265 | 9265 | 35801208 | Comfy 2 bedroom<br>Close To<br>Manhattan  | 256911412 | Taylor | E<br>p   |

| Unnamed: 0 |      | listing_id | name                                | host_id | host_name           | room_t |
|------------|------|------------|-------------------------------------|---------|---------------------|--------|
| 1145       | 1145 | 253806     | Loft Suite @ The Box House Hotel    | 417504  | The Box House Hotel | En p   |
| 6562       | 6562 | 253806     | Loft Suite @ The Box House Hotel    | 417504  | The Box House Hotel | En p   |
| 8699       | 8699 | 2044392    | The heart of Williamsburg 2 bedroom | 620218  | Sarah               | En p   |
| 5761       | 5761 | 2044392    | The heart of Williamsburg 2         | 620218  | Sarah               | En p   |